SOFTWARE REQUIREMENTS SPECIFICATION

MUSHIN 2.0

AI-Powered Creator Intelligence Platform

PROJECT	MUSHIN
SRS VERSION	1.0 — Final Draft
PREPARED BY	Ali Saad Khan (14757) Shayan Ali Khan (14632)
SUPERVISOR	Sir Mumtaz Ali
PROGRAM	BS Software Engineering — Final Year Project
DATE	September 23, 2026

“Helping brands find relevant and authentic creators based on their requirements, saving hours of manual searching and verification.”

Template based on IEEE SRS format (Karl E. Wiegers, 1999). Document content © 2026 MUSHIN.

Table of Contents

Revision History 2
1. Introduction 3
1.1 Purpose 3
1.2 Document Conventions 3
1.3 Intended Audience and Reading Suggestions 3
1.4 Product Scope 3
1.5 References 4
2. Overall Description 5
2.1 Product Perspective 5
2.2 Product Functions 6
2.3 User Classes and Characteristics 6
2.4 Operating Environment 7
2.5 Design and Implementation Constraints 7
2.6 User Documentation 7
2.7 Assumptions and Dependencies 7
3. External Interface Requirements 8
3.1 User Interfaces 8
3.2 Hardware Interfaces 8
3.3 Software Interfaces 9
3.4 Communications Interfaces 9
4. System Features 10
4.1 Creator Intelligence 10
4.2 Search and Discovery 11
4.3 AI Intelligence Layer 12
4.4 Lists and CRM 13
4.5 Campaign Management 14
4.6 Outreach and Communication 14
4.7 Workspace and Collaboration 15
4.8 Billing and Credits 16
4.9 Analytics 17
4.10 Administration 17
5. Other Nonfunctional Requirements 18
5.1 Performance Requirements 18
5.2 Safety Requirements 18
5.3 Security Requirements 19
5.4 Software Quality Attributes 19
5.5 Business Rules 20
6. Other Requirements 20
6.1 Database Requirements 20
6.2 Internationalization 20
6.3 Legal and Compliance 20

6.4 Data Retention 20
Appendix A: Glossary ..... 21 Appendix B: Analysis Models 22
Appendix C: To Be Determined List 23

Revision History

Name	Date	Reason for Changes	Version
Ali Saad Khan; Shayan Ali Khan	2026-09-23	Initial consolidated SRS based on the approved MUSHIN project requirements and specifications.	1.0

Document version: 1.0 — Final Draft

Project: MUSHIN 2.0 | Final Year Project — BS Software Engineering

1.1 Purpose

This document is the Software Requirements Specification (SRS) for MUSHIN 2.0, an AI-powered Creator Intelligence and Influencer Marketing Platform. It covers Version 1.0 of the product, the Stage 1 (S1) release. This SRS defines what the software must do and serves as the main requirements reference for developers, designers, testers, and business stakeholders throughout the project.

1.2 Document Conventions

The following conventions are used throughout this document:

- Bold text marks key terms or requirement IDs on their first use.
- Requirement IDs follow the format REQ-XX.YY where XX is the feature area number and YY is the requirement number within that area.
- Priority levels used are: Must Have (M), Should Have (S), and Could Have (C).
- MVP = Minimum Viable Product — the smallest working version that can be released to early users.
- S1 = Stage 1 (MVP launch), S2 = Stage 2 (public launch), S3 = Stage 3 (enterprise launch).
- TBD = To Be Determined — items not yet finalized are marked TBD and listed in Appendix C.
- Requirements in this document are derived from the approved MUSHIN project requirements and specifications.

1.3 Intended Audience and Reading Suggestions

This document is written in plain language for multiple types of readers:

Developers and Engineers

Read all sections. Pay special attention to Section 4 (System Features) and Section 5 (Nonfunctional Requirements). Section 3 (External Interface Requirements) describes how the system connects to outside services.

Product Managers and Business Owners

Start with Section 1.4 (Product Scope) and Section 2 (Overall Description). Section 4 gives the full feature list organized by area. Section 5.5 (Business Rules) explains important rules about how the product works.

QA Testers

Section 4 lists all functional requirements by feature. Section 5 describes performance, security, and quality targets. Use these to create test cases.

Designers

Section 2.3 (User Classes) and Section 3.1 (User Interfaces) are most relevant. Section 2.2 (Product Functions) gives a high-level picture of what needs UI.

Investors and Stakeholders

Section 1.4 (Product Scope) and Section 2.1 (Product Perspective) give the business context. Section 4 shows what will be built.

1.4 Product Scope

MUSHIN 2.0 is a web-based platform that helps marketing teams and agencies find, vet, and work with social media creators (influencers). Today, brands use 4–6 separate tools just to run one campaign — a database to find creators, a separate tool to check for fake followers, an email tool to contact them, a spreadsheet to track progress, and a reporting tool at the end. MUSHIN replaces all of these with one connected platform.

1.5 References

The platform's main value is intelligence over inventory. Instead of competing by having the biggest database of creators,

MUSHIN focuses on ranking creators by campaign fit, explaining authenticity signals, and preserving collaboration history. The first version targets marketing agencies and direct-to-consumer brands, launching first in Pakistan with global

capability.

External references

[1] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” 2018. https://www.w3.org/TR/WCAG21/

[2] IETF, E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, 2018. https://www.rfc-editor.org/rfc/rfc8446

[3] IETF, “The OAuth 2.0 Authorization Framework,” RFC 6749, 2012. https://www.rfc-editor.org/rfc/rfc6749

[4] PostgreSQL Global Development Group, “PostgreSQL Documentation.” https://www.postgresql.org/docs/

[5] Amazon Web Services, “Amazon Simple Queue Service (Amazon SQS) Documentation.” https://docs.aws.amazon.com/sqs/

[6] Cloudflare, “Cloudflare R2 Documentation.” https://developers.cloudflare.com/r2/

[7] Apify, “Apify Documentation.” https://docs.apify.com/

[8] Serper, “Serper API Documentation.” https://serper.dev/

[9] OpenAI, “API Documentation.” https://platform.openai.com/docs/

[10] Paddle, “Paddle Developer Documentation.” https://developer.paddle.com/

[11] Google, “Gmail API Documentation.” https://developers.google.com/gmail/api

[12] Microsoft, “Microsoft Graph Documentation.” https://learn.microsoft.com/graph/

[13] Meilisearch, “Meilisearch Documentation.” https://www.meilisearch.com/docs/

2.1 Product Perspective

MUSHIN 2.0 is a new, standalone product. It is not a replacement for an existing system — it is a brandnew platform designed to consolidate a fragmented market into one place. The influencer marketing industry is currently served by several disconnected tools: creator databases (like Modash or HypeAuditor), email tools, CRM spreadsheets, and analytics dashboards. Brands typically stitch 4 to 6 of these tools together per campaign, losing time and information at every handoff.

MUSHIN sits in the middle of all of this as the single platform for the entire creator relationship lifecycle. It connects to external services (Google/Instagram/TikTok public data via web scraping, Gmail and Outlook for email, and Paddle for payments) through a structured adapter layer that acts as the only gateway to outside services. The platform's two-part data engine ("Two Brains") first searches its own growing database of creator profiles instantly, then — if results are not enough — discovers new creators live by scraping public social media data. Every live discovery permanently adds to the database, making future searches faster and cheaper over time.

2.2 Product Functions

At a high level, MUSHIN 2.0 gives users the ability to:

- Search and discover creators using plain-English queries, filters, or similarity to creators they already know.
- View detailed creator profiles including audience size, engagement rates, content topics, and an AIgenerated authenticity score that explains exactly why a creator might have fake followers.
- Organize creators into lists and collaborate with team members with comments and shared access.
- Plan and run marketing campaigns with pipelines, tasks, budgets, and delivery tracking.
- Contact creators directly through connected Gmail or Outlook accounts with templates, follow-up sequences, and reply tracking.
- Track campaign results and generate reports for clients or stakeholders.
- Manage team members with different permission levels within workspaces.
- Buy subscriptions and credits to control spending on AI and data features.
- Access an internal admin panel (for MUSHIN staff) to support users and monitor the platform.

2.3 User Classes and Characteristics

MUSHIN 2.0 has five types of users, ordered by how important they are to the initial launch:

"Agency Alex" — Account Director at an Influencer Marketing Agency

Alex runs 10 or more client campaigns at once and needs to defend every creator choice with real data. He has 12 people on his team. His biggest pain points are: spending hours building shortlists that are hard to explain to clients, having campaign notes scattered across spreadsheets, and not being able to remember

what rates creators quoted last time. Alex is the most important user for the product's initial version. He uses: Search, Lists, Campaigns, Analytics, Workspace features.

"Brand Bianca" — Influencer Marketing Manager at a DTC Brand

Bianca manages creator partnerships for a brand doing about $20M in revenue. She needs her campaigns to hit specific cost and return targets. She was burned once by a campaign where the creator turned out to have mostly fake followers, wasting a significant budget. She is the second most important user for the initial version. She uses: Creator profiles (especially authenticity scores), AI features, Campaign management, Outreach, Analytics.

"Growth Gabe" — Startup Growth Lead

Gabe needs to test influencer marketing quickly on a tight budget. He cannot afford expensive enterprise tools. The credit-based pricing model works well for him because he only pays for what he actually uses. His most important journey is signing up, doing a search, and getting to a useful result within 5 minutes. He uses: Search, Lists, AI features, Billing (credits).

"Enterprise Elena" — Head of Influencer at a Global Consumer Brand

Elena manages influencer marketing across multiple regions and teams. She needs governance, audit trails, and brand safety features. She is not the target for the first version (S1) because enterprise features like SSO and deep audit logs are planned for Stage 3. She uses (eventually): All features plus advanced RBAC, SSO, compliance reporting.

"Manager Mia" — Talent/Creator Manager

Mia manages a roster of 25 creators and wants to track inbound opportunities and deal progress. She is a secondary user — the platform serves her needs through the same features built for the above users, with no special features built just for her at launch. She uses: Lists, Campaigns, Outreach.

2.4 Operating Environment

- The platform runs as a web application (browser-based). There is no mobile app for S1 or S2. The site is designed to work on all modern browsers on desktop screens.
- The application is hosted on Vercel (web server and API), with the database on Neon/Supabase PostgreSQL (a managed cloud database service).
- Background work (like scraping new creator data) runs on Railway — a separate server platform suited for long-running tasks.
- Files and raw data archives are stored in Cloudflare R2 (cloud object storage).
- Creator search uses Meilisearch Cloud as a fast search engine that sits in front of the main database.
- Task queuing uses AWS SQS FIFO, which handles background jobs in an orderly, reliable way.
- Upstash Redis is used for rate limiting and caching of frequently-requested data like entitlement checks.

2.5 Design and Implementation Constraints

- All code is written in TypeScript (strict mode). No use of the "any" type is allowed.
- The backend is a modular monolith — one codebase, but divided into clear sections (modules) that each own their own data and logic. Workers (background jobs) run as separate processes from the same codebase.
- All communication with external services (Apify, Serper, OpenAI, Gmail, Outlook, Paddle) must go through a dedicated Adapter. No direct API calls from business logic.
- Payment and billing are handled entirely by Paddle (Merchant of Record). MUSHIN never stores card data or generates tax invoices.
-

Database access for workspace data must always filter by workspace ID. This rule is enforced at three layers: the application middleware, the database repository layer, and PostgreSQL Row Level Security (RLS).

- Every time a credit-spending action runs, it must record exactly how much it cost. This is non-negotiable for margin control.
- Events are published through a transactional outbox. API handlers never write directly to the queue.
- The platform launches in Pakistan first. Creator data coverage for Pakistani and South Asian creators must be validated before S1 launch.
- No feature may be built that bypasses the above rules, even temporarily. There are no exceptions.

2.6 User Documentation

The following documentation will accompany the software:

- In-app onboarding guide — step-by-step walkthrough of the Golden Path (sign up !' search !' enrich one creator !' save a list). Shown on first login.
- Help center articles — written guides for each major feature area (Search, CRM, Campaigns, Outreach, Billing).
- Credit usage guide — explains how credits are spent, how to buy more, and how to see your usage history.
- API documentation — for any future public API (not part of S1; planned for S3).

2.7 Assumptions and Dependencies

- It is assumed that Apify (the scraping service) can reliably collect public profile data for Instagram, TikTok, and YouTube creators, including Pakistani and South Asian creators. This must be validated with a test panel before launch.
- It is assumed that Serper (Google search API) can surface relevant creator candidates when given a search query. Quality must be tested before launch.
- It is assumed that OpenAI or a compatible LLM provider can analyze raw scraped data and produce accurate authenticity scores and summaries at an acceptable cost per action.
- Paddle must be able to onboard a Pakistan-registered entity. This is an existential dependency (ADR-032). If it fails, an alternative entity structure must be found before launch.
- Gmail and Outlook APIs must allow the outreach sequence automation planned by MUSHIN, within their Terms of Service.
- All audience demographic data (age, gender, location breakdown) shown on creator profiles will be estimated by AI from public signals, not measured directly. These estimates must be clearly labeled as estimates in the product UI.

3.1 User Interfaces

The application is a responsive web app designed for desktop browsers. The key UI requirements are:

- Every screen loads within the workspace context. The user always sees which workspace they are in (top-left workspace switcher).
- The left sidebar has exactly six navigation items: Search, Lists, Campaigns, Inbox, Analytics, Settings. These labels are fixed — no synonyms.
- A credit meter is always visible in the top-right corner showing the user's current balance. It turns amber when credits are running low, and red when very low.
- A notification bell/tray is always visible in the top bar.
- A global search bar (omnibar) at the top lets users look up any creator by name, handle, or URL.
- Every empty state (no search results, no campaigns yet, empty list) must show the user what to do next. Dead-ends are not allowed.
- When an action costs credits, the user must see the estimated cost before confirming. They must never be surprised by a charge.
- Long-running background jobs (like a live creator discovery) must show visible progress. If a job takes more than 30 seconds, the user gets a notification when it is done instead of having to watch a loading spinner.
- AI scores and estimates (authenticity scores, audience estimates) must always show the evidence behind them. A score without explanation is not allowed in the product. WCAG 2.1 AA
- accessibility compliance is required for Stage 2. [1]

3.2 Hardware Interfaces

MUSHIN 2.0 is a cloud-hosted web application. It has no direct hardware interfaces. Users access it through a standard web browser on any device with internet access. The platform does not connect to any specialized hardware components. The underlying infrastructure (servers, storage, network) is fully managed by the cloud providers listed in Section 2.4.

3.3 Software Interfaces

MUSHIN 2.0 connects to the following external software services:

Apify — Web Scraping Platform

Used to collect public creator profile data from Instagram, TikTok, and YouTube. MUSHIN uses Apify's managed actors (pre-built scraping programs). The connection uses Apify's REST API with API token authentication. Data flows from Apify into MUSHIN's internal database through the Standardization module.

Serper — Google Search API

Used to find new creator candidates by running structured Google searches based on a user's query. Results (URLs and page summaries) are fed into the scraping pipeline. Results are cached for 7 days to reduce costs.

OpenAI / LLM Provider

Used to analyze scraped creator data and produce: authenticity scores, audience summaries, creator briefs, and search query interpretation. The system uses a tiered approach: a cheaper model by default, a more powerful one only when needed. Every LLM call records its cost.

Paddle — Payment Processing

Handles all subscription purchases, credit top-ups, invoicing, tax calculation, and payment compliance. MUSHIN never stores card data. All billing state is driven by Paddle webhooks. Paddle is the Merchant of Record — the legal seller of the product.

Gmail / Google OAuth [11]

Allows users to connect their personal or work Gmail account. MUSHIN reads and sends emails on behalf of the user for creator outreach. The connection uses OAuth 2.0. Only the connected account owner can use their mailbox through MUSHIN.

Outlook / Microsoft OAuth [12]

Same as Gmail but for Microsoft/Outlook email accounts. The same consent and security rules apply.

Meilisearch Cloud — Search Index

A fast search engine that indexes all creator profiles. When a user searches, MUSHIN first queries Meilisearch for instant results before considering any live scraping. Only global creator data goes into the index — no workspace-private data.

PostgreSQL (Neon / Supabase)

The main relational database. Stores all creator data (global) and all workspace data (lists, campaigns, notes, ledger) in two strictly separated schemas. Row Level Security is enabled on all workspace tables.

AWS SQS

The message queue for background job processing. Background tasks (scraping, enrichment, email delivery) are sent to queues and picked up by worker processes. Uses FIFO ordering with separate queues for different priority levels.

Cloudflare R2 — Object Storage

Stores raw scrape payloads (archived for reprocessing) and GDPR export bundles. Files attached to CRM records are also stored here.

3.4 Communications Interfaces

- All traffic between the browser and the MUSHIN server uses HTTPS (TLS 1.2 or higher). HTTP is not supported. [2]
- The backend API communicates with external services over HTTPS using their official REST APIs.
- Paddle sends payment event notifications to MUSHIN via webhooks over HTTPS. The signature of every incoming webhook is verified before processing begins.
- Apify job results are delivered back to MUSHIN by polling the Apify API or receiving a webhook callback.
- Email is sent through the user's connected Gmail or Outlook account — MUSHIN does not have its own email-sending infrastructure for outreach.
-
-
-

Transactional emails from MUSHIN to users (notifications, credit alerts) are sent via Resend or Postmark.

Real-time job progress updates are delivered to the browser using a lightweight polling mechanism in S1. Push-based delivery (WebSockets) is planned for a later stage.

Internal services (API server to worker fleet) communicate only through the message queue (AWS SQS). They do not call each other directly over HTTP.

The ten system features below map directly to the ten core pillars of the MUSHIN product.

requirements it must satisfy.

4.1 Creator Intelligence

4.1.1 Description and Priority

Creator Intelligence is the core unit of value in MUSHIN. It is a detailed profile for each creator (influencer) that combines data from multiple social media platforms, AI analysis, and enrichment from public web sources. Without high-quality creator profiles, nothing else in the platform works. Priority: High (Must Have for S1).

4.1.2 Stimulus / Response Sequences

- User searches for a creator !' System shows a result card with a summary of the creator's data.
- User clicks a creator card !' System opens the full creator profile page with all available data.
- User clicks "Enrich" on a profile !' System shows the estimated credit cost !' User confirms !' Background job scrapes updated data !' Profile is refreshed with new information.
- User views an authenticity score !' System shows the score AND the evidence behind it (specific signals like follower-growth spikes, comment patterns, etc.).
- User clicks "Reveal Contact" !' System deducts credits !' Creator's email/social links are revealed.

4.1.3 Functional Requirements

REQ-01.01	The system must show a unified creator profile that combines data from multiple social platforms (Instagram, TikTok, YouTube). Data includes: follower count, engagement rate, growth history, posting frequency, content topics, and niche category. (Must Have / S1)
REQ-01.02	The system must provide an explainable authenticity score for every creator. The score is never shown without the evidence that produced it (e.g., "High follower spike in March 2024 with no viral content detected"). (Must Have / S1)
REQ-01.03	The system must support credit-gated contact reveal: clicking "Reveal" deducts credits and shows the creator's email and social profile links. (Must Have / S1)
REQ-01.04	The system should show historical metrics over time (follower growth chart, engagement trends) and any past brand collaboration history found in public data. (Should Have / S2)
REQ-01.05	The system should show content insights: what topics the creator posts about most, what formats perform best, and typical performance patterns. (Should Have / S2)
REQ-01.06	Every creator profile must show a "freshness indicator" showing when the data was last updated. Users can trigger a manual refresh (costs credits). (Must Have / S1)
REQ-01.07	Audience demographic data (age, gender, location) must be clearly labeled as "estimated" — it is modeled from public signals, not measured directly. (Must Have / S1)

4.2 Search and Discovery

4.2.1 Description and Priority

Search is the first thing users do in MUSHIN and the fastest way to prove value. It must be fast, smart, and handle cases where the internal database does not yet have the creators a user is looking for. Priority: High (Must Have for S1).

4.2.2 Stimulus / Response Sequences

- User types filters (platform: Instagram, location: Pakistan, followers: 10k–500k) !' System returns ranked results in under 1 second.
- User types a plain-English query ("Pakistani skincare creators with real engagement") !' System interprets the query into search filters !' Shows ranked results.
- Results are fewer than 8 or poor quality !' System shows existing results AND a "Search Live" option with a credit cost shown !' User confirms !' Background discovery job runs !' New creator results appear progressively as they are found.
- User adds a creator URL directly !' System looks up or scrapes that specific creator !' Profile is created or refreshed.
- User saves a search !' System stores the search parameters !' User can re-run it later with one click.

4.2.3 Functional Requirements

REQ-02.01	The system must support advanced filtered search with at minimum these filter dimensions: platform, audience location, creator location, follower size range, engagement rate range, content category/niche. Results must return in under 1 second (p95). (Must Have / S1)
REQ-02.02	The system must support natural-language search queries. The system interprets the query into editable filter chips that the user can adjust before or after seeing results. (Must Have / S1)
REQ-02.03	Search Results must be ranked by fit-relevance to the query, not just by follower count or random order. (Must Have / S order.
REQ-02.04	Users should be able to save searches and re-run them with one click. (Should Have / S1)
REQ-02.05	The system should support similarity search: given one creator, find others with similar content, audience, or style. (Should Have / S2)
REQ-02.06	The system could show an explanation for why a creator ranked in a particular position ("why this creator is #1"). (Could Have / S2)
REQ-02.07	When the live discovery path is triggered, the user must always explicitly confirm before credits are reserved. The system never automatically starts a credit-spending operation without user action. (Must Have / S1)
REQ-02.08	The Golden Path onboarding must work even if the database has zero creators in the user's niche. The system must guide the user to the add-by-URL path as a fallback. (Must Have / S1)

4.3 AI Intelligence Layer

4.3.1 Description and Priority

AI features appear throughout the platform — they are not a separate section but a layer of intelligence on top of everything else. Every AI feature must be explainable, reversible, and cost-tracked. Priority: High (Must Have for governance rules in S1; individual AI features vary).

4.3.2 Stimulus / Response Sequences

- User opens a creator profile !' System automatically generates a 3-sentence AI summary of the creator, readable without any data analysis by the user.

-

- User types a complex search query !’ AI interprets it into structured filters, which the user can edit.
- User drafts an outreach message !’ AI suggests a personalized version based on the creator's content style (optional, user can ignore).
- Admin sees cost dashboard !’ AI feature costs are broken down per action, per provider, per day.

4.3.3 Functional Requirements

REQ-03.01	The system must generate an AI summary for each creator profile — a short, plain-English brief that a marketer can read in 10 seconds to decide if this creator is worth investigating further. (Must Have / S1)
REQ-03.02	The system should assist users in constructing search queries from vague descriptions. (Should Have / S1)
REQ-03.03	The system should offer AI-assisted outreach message drafting, personalised to the specific creator and the user's brand voice. (Should Have / S2)

REQ-03.04	The system could send alerts when a tracked creator shows sudden signs of suspicious activity (bot-like follower spikes, unusual engagement drops). (Could Have / S2)
REQ-03.05	Every AI feature must record its cost per action. No AI endpoint can be deployed without cost instrumentation. (Must Have / S1 — governance rule)
REQ-03.06	AI outputs must always be presented as suggestions or estimates, not facts. Users must be able to override or ignore any AI output. No AI action should automatically do something irreversible without user confirmation. (Must Have / S1 — governance rule)

4.4 Lists and CRM

4.4.1 Description and Priority

Lists are how users organize creators they are interested in. The CRM layer builds relationship memory over time — notes, history, and context that follow a creator across campaigns. Priority: High (Must Have for S1).

4.4.2 Stimulus / Response Sequences

- User selects creators from search results and clicks "Add to List" !' System adds them to a named list.
- User opens a list !' Sees all creators with their data summary and any notes added by the team.
- User adds a note to a creator in a list !' Note is timestamped and visible to all list collaborators.
- User shares a list with a colleague !' Colleague can view (and optionally comment/edit) the list.
- User exports a list to CSV !' System checks entitlement !' If allowed, downloads a file with creator data.

4.4.3 Functional Requirements

REQ-04.01	Users must be able to create named lists, add creators from search results (individually or in bulk), and write notes and add tags to creators in a list. (Must Have / S1)
REQ-04.02	Lists must support collaboration: sharing with specific workspace members, viewing who added notes or made changes, and leaving comments. (Must Have / S1)
REQ-04.03	Users should be able to export a list to CSV. Exports are gated by subscription tier or credit spend. Users can also attach files to creator records. (Should Have / S1)
REQ-04.04	The system should build relationship memory for each creator: rate history, past campaign outcomes, interaction log — all visible on the creator's profile within the workspace context. (Should Have / S2)
REQ-04.05	Users could perform bulk actions on a list: move creators between lists, add them to a campaign, or trigger bulk enrichment with a total cost preview shown before confirmation. (Could Have / S2)

4.5 Campaign Management

4.5.1 Description and Priority

Campaign Management turns MUSHIN from a research tool into a full operating system for influencer marketing. It tracks every creator through the campaign lifecycle. Priority: High (Must Have core features for S1).

4.5.2 Stimulus / Response Sequences

- User creates a new campaign, fills in a brief (goals, target audience, budget) !' Campaign is created and appears in the campaigns list.
- User adds creators to the campaign pipeline !' Each creator gets a status card (e.g., "Contacted," "Negotiating," "Confirmed").
- User moves a creator card to the next stage !' Status updates, and a timeline event is recorded.
- User views campaign analytics !' Sees spend vs. budget, status of each creator, deliverable count.

4.5.3 Functional Requirements

REQ-05.01	Users must be able to create a campaign with a structured brief including: name, description, target audience, content requirements, budget, and start/end dates. (Must Have / S1)
REQ-05.02	Campaigns must have a pipeline view with customizable stages. Each creator in the campaign has a status card that can be moved between stages. (Must Have / S1)
REQ-05.03	Users should be able to create tasks and milestones within a campaign, assign them to team members, and track budget spent per creator. (Should Have / S1)
REQ-05.04	Users should be able to track deliverables (posts, videos, stories) and mark them as approved or pending. (Should Have / S2)
REQ-05.05	Campaign performance analytics (ROI, spend vs. results) should be available as a dashboard view. (Should Have / S2)
REQ-05.06	Contract documents could be stored within a campaign as file attachments. E-signature is not built in — it requires an external tool. (Could Have / S3)

4.6 Outreach and Communication

4.6.1 Description and Priority

Outreach allows users to contact creators directly from MUSHIN using their own Gmail or Outlook account, with templates and automated follow-up sequences. Priority: High (Must Have for S1).

4.6.2 Stimulus / Response Sequences

- User connects Gmail via OAuth !’ Mailbox is linked to their MUSHIN account.
- User selects a creator and clicks "Send Email" !’ Selects a template !’ Edits the draft !’ Sends.
- User creates a sequence (initial email + 2 follow-ups with delays) !’ System sends follow-ups automatically, stops if the creator replies.
- User views the campaign inbox !’ Sees all email threads with creators, linked to the correct campaign.

4.6.3 Functional Requirements

REQ-06.01	The system must support connecting Gmail and Outlook accounts via OAuth. Emails sent and received appear linked to the creator's profile and the relevant campaign. (Must Have / S1)
REQ-06.02	Users must be able to create and use email templates with personalization variables (creator name, platform handle, etc.). (Must Have / S1)
REQ-06.03	Users should be able to create multi-step sequences (timed follow-ups). Sequences must stop automatically when a creator replies. (Should Have / S1)
REQ-06.04	Users should be able to see open/reply tracking and a shared campaign inbox showing all creator conversations. (Should Have / S2)
REQ-06.05	Users could schedule emails for a specific send time and add internal notes visible only to their team on email threads. (Could Have / S2)
REQ-06.06	The system must respect consent: if a creator has opted out of outreach (marked in the system), they must be blocked from future sequence enrollment. (Must Have / S1)

4.7 Workspace and Collaboration

4.7.1 Description and Priority

Workspaces are the containers that keep each client's or team's data completely separate. Roles control what each team member can do. Priority: High (core workspace isolation is Must Have for S1).

4.7.2 Stimulus / Response Sequences

- New user signs up !' Creates a workspace !' Gets Owner role !' Invites teammates by email.
- Invited teammate accepts !' Gets assigned a role (Admin or Member) !' Can see and work on all workspace data appropriate to their role.
- Workspace owner switches between two workspaces !' All data, lists, campaigns, and settings change to the selected workspace. No data from Workspace A is visible in Workspace B. Admin views billing !' Can
- see current subscription, credit balance, and usage history.

4.7.3 Functional Requirements

REQ-07.01	Users must be able to create workspaces, invite members by email, and assign them roles: Owner, Admin, or Member. (Must Have / S1)
REQ-07.02	All workspace data must be completely isolated. A user in Workspace A can never see or access data from Workspace B. This isolation is enforced at the database level (Row Level Security), not just in the application code. (Must Have / S1)
REQ-07.03	Users should have granular role-based permissions, an activity history showing recent actions, and in-app notifications for relevant events. (Should Have / S2)
REQ-07.04	The system could support SSO (Single Sign-On using SAML), exportable audit logs, and data processing agreement workflows. These are for enterprise customers and are planned for Stage 3. (Could Have / S3)

4.8 Billing and Credits

4.8.1 Description and Priority

Billing is handled through Paddle. Credits are the internal currency for AI and data features. The system must be accurate, transparent, and impossible to game. Priority: High (Must Have for S1 — no revenue without it).

4.8.2 Stimulus / Response Sequences

- User clicks "Subscribe" !' Paddle-hosted checkout opens !' User pays !' Paddle sends a webhook !' MUSHIN's system updates the user's subscription state and credit balance.
- User performs a credit-spending action !' System checks credit balance !' Reserves the credits !' Runs the action !' If successful, commits the spend. If the action fails due to a system error, the reserved credits are released.
- Subscription is cancelled !' Paddle sends a webhook !' System downgrades the workspace to a free/ expired state !' User sees a clear message about what they have lost access to.

4.8.3 Functional Requirements

All subscription purchases and credit top-ups must go through a Paddle-hosted checkout. MUSHIN never processes payments directly. (Must Have / S1)

A credit ledger must record every credit transaction: allowance granted, credits spent, credits reserved, and credits released. Users can see their full ledger history. The ledger is append-only — entries are never changed or deleted. (Must Have / S1)

Before any credit-spending action runs, the system must reserve the credits (locking them), run the action, then either commit or release the reservation. Credits are never silently lost due to a failed action. (Must Have / S1)

REQ-08.04	Trial accounts must have a hard credit cap so that free usage cannot exceed a controlled cost limit. Trial expiry must be handled gracefully with a clear upgrade prompt. (Must Have / S1)
REQ-08.05	Users should see a credit usage dashboard showing what they have spent credits on. The system should handle payment failures gracefully, giving users time to update their payment method before losing access. (Should Have / S2)

4.9 Analytics

4.9.1 Description and Priority

Analytics shows users whether their campaigns are working and helps them prove the value of influencer marketing to clients or management. Priority: Should Have for S1 (basic); Higher depth in S2.

4.9.2 Stimulus / Response Sequences

- User opens a campaign !' Clicks "Analytics" tab !' Sees current status of each creator, total spend, number of deliverables completed.
- User wants a client report !' Exports a PDF or data export of campaign results.

4.9.3 Functional Requirements

REQ-09.01	The system should show a basic campaign performance dashboard: creator statuses, total spend vs. budget, deliverable completion count. (Should Have / S1)
REQ-09.02	Users should be able to generate an exportable client report showing campaign results. (Should Have / S2)
REQ-09.03	The system could show workspace-level analytics (outreach response rates, total creators contacted) and eventually cross-campaign ROI modeling. (Could Have / S2–S3)

4.10 Administration (Internal)

4.10.1 Description and Priority

The admin panel is used by MUSHIN's own staff to support customers, monitor the platform's health, and manage feature flags. It is not visible to customers. Priority: High (Must Have for S1 — without it, support is impossible).

4.10.2 Stimulus / Response Sequences

- MUSHIN support agent searches for a customer !' Views their workspace, subscription state, and recent activity.
- Support agent impersonates a customer (with the customer's knowledge) to debug an issue !' Every action taken during impersonation is logged separately.
- Engineer turns on or off a feature flag !' Specific feature becomes available/unavailable for selected workspaces.

4.10.3 Functional Requirements

REQ-10.01	The admin panel must allow MUSHIN staff to look up users and workspaces, impersonate a user session (for support purposes), and toggle feature flags. All impersonation sessions must be recorded in an audit log. (Must Have / S1)
REQ-10.02	The system must show MUSHIN staff a provider monitoring dashboard: current API usage, costs, and error rates for each external service (Apify, Serper, LLM providers). (Must Have / S1)
REQ-10.03	The system should provide revenue and credit usage views for operations, moderation tools for abuse, and automated alerts when something looks wrong. (Should Have / S2)

5.1 Performance Requirements

These targets must be met under normal expected load. "p95" means 95% of requests must complete within the stated time — the worst 5% can be slower.

NFR-P01	Filtered search response time	*Under 1 second (p95)*
NFR-P02	Natural-language / semantic search time	*Under 3 seconds (p95), with results loading progressively*
NFR-P03	Known creator data refresh	*User sees progress; completes within 30 seconds (p95)*
NFR-P04	Live discovery job (new creators)	*Async — no time limit; user notified on completion. Progressive results shown*
NFR-P05	General page load time	*Under 2 seconds (p95) for all non-search pages*
NFR-P06	Credit balance check	*Real-time; no perceptible delay on credit-gated action screens*

5.2 Safety Requirements

MUSHIN handles sensitive business data (campaign strategy, creator contacts, rate negotiations) and personal data of creators (public figures). The following safeguards must be in place:

- Creators' personally identifiable information (email addresses, phone numbers) is only revealed when a user explicitly pays credits to reveal it. It is not shown by default.
- Credit-spending actions are protected by a reserve-commit pattern: if the system crashes mid-action, credits are not permanently lost. A cleanup job restores them.
- No AI-generated action (sending an email, enrolling in a sequence) can happen automatically without a human clicking a confirmation. Irreversible actions always require explicit user approval.
- All creator data is scraped from public sources only. No private or logged-in content is ever accessed.
- All raw scraped data is archived for auditability. If a score is ever disputed, it can be reconstructed from the source data.
- Minor-flagged creators (where the system has detected signals of being under 18) are blocked by default from contact reveal, campaign enrollment, and outreach. This is a safety default, not a configurable setting.

5.3 Security Requirements

Security is non-negotiable and applies regardless of effort or timeline pressure:

- Authentication is provided by a managed auth service (not custom-built). Passwords are never stored in plain text — they are managed by the auth provider.
- Sessions use short-lived JWT tokens (15 minutes) plus rotating refresh tokens. A compromised refresh token can be detected and the entire session family revoked.
- Multi-factor authentication (TOTP) is optional for users at S1, required for workspace Owners at S2.
- Workspace data isolation is enforced at three independent layers: application middleware, database repository code, and PostgreSQL Row Level Security. A bug in any one layer cannot cause a data leak because the other two layers catch it.
- MUSHIN staff access (admin panel) is on a completely separate authentication system. Staff roles are deny-by-default: staff cannot do anything without an explicit action, and every action is logged.
- No credentials are stored in code. All secrets are injected at runtime from a managed secret store.
- Payment card data never touches MUSHIN systems. Paddle handles all of it.
- Scraped content is treated as hostile input. It is processed in a sandboxed pipeline and cannot affect the search results shown to other users (write-path / read-path separation).
- All sensitive data in transit is encrypted with TLS 1.2 or higher.
- Rate limiting is applied at the API boundary to prevent abuse.

5.4 Software Quality Attributes

Reliability

The platform must maintain 99.9% uptime for customer-facing features from Stage 2 onward. Background jobs (enrichment, discovery) can fail and retry without user-visible impact as long as the user is notified of the outcome.

Maintainability

The codebase is a modular monolith. Each module owns its own data and logic. No module may directly access another module's database tables. This makes it safe to change one part of the system without accidentally breaking another. All code must be in TypeScript (strict mode).

Testability

Every module must have: unit tests (happy path, error cases, edge cases), integration tests (API contracts, database writes, event emission), and security tests (cross-tenant access attempts, permission checks). The test suite must catch regressions before code reaches production.

Observability

Every request must carry a unique request ID. Logs must be structured JSON (machine-readable). Every external API call must record its cost. Alerts must fire within 1 hour if any provider's API spending doubles vs. its baseline.

Scalability

The modular monolith and worker fleet are designed to scale independently. The web tier (Vercel) scales on request volume; the worker fleet (Railway) scales on queue depth. The architecture is designed so that the most demanding module (Live Discovery) can be extracted into its own service if needed without a full rewrite.

Usability

The interface follows progressive disclosure: basic users never see enterprise-level complexity unless they need it. Key tasks for the primary personas (Alex and Bianca) must be completable without reading documentation. The "time to first aha" — from sign-up to seeing a ranked search result for a creator — must be under 5 minutes.

5.5 Business Rules

- Credits are the only unit of metered consumption. They cannot be earned — only purchased via subscription allowances or credit top-up packs.
- Every credit-priced action must cost at least 3x its raw API cost (the "3x COGS rule"). This is a nonnegotiable margin protection rule for all new features.
- Subscription entitlements (what the workspace is allowed to do) are always derived from Paddle's record of the subscription. MUSHIN's local copy is a cache — Paddle is always right in a conflict.
- Creators flagged as potential minors are blocked from contact reveal, outreach enrollment, and campaign addition by default. This cannot be overridden by a workspace Owner — it requires MUSHIN staff action.
- The ledger is append-only. A mistake in the ledger is corrected by adding a new correction entry, never by editing or deleting an existing entry.

- AI scores must be labeled as estimates and must include the evidence that produced them. A score displayed without evidence is a product defect.
- Users may only send outreach from their own connected mailbox. Sending as another user is not possible even with admin-level permissions.

6.1 Database Requirements

The database uses PostgreSQL with two strictly separated schemas:

- The Global Creator Plane (GCP) holds creator and profile data shared across all workspaces. It has no workspace-specific data.
- The Workspace Plane (WP) holds all tenant-specific data (lists, campaigns, notes, ledger, etc.). Every table in this schema has a workspace_id column. Row Level Security policies on all WP tables enforce isolation at the database engine level.
- Database migrations follow the expand-contract pattern: columns and tables are added first, old ones are deprecated later, and nothing is ever dropped without a long deprecation window. DROP COLUMN and DROP TABLE operations are forbidden on production data.
- The ledger and timeline tables are append-only. No update or delete operations on these tables are permitted — ever.

6.2 Internationalization

- The first version (S1) is English-only for the user interface.
- Creator search must handle transliteration — a Pakistani creator with a name spelled multiple ways in Latin script (or partially in Urdu) must still appear in results.
- Urdu language UI is logged as a future consideration for S2 or later.
- Currency handling for campaign budgets must support multiple currencies with FX conversion via a dedicated FX adapter.

6.3 Legal and Compliance

- MUSHIN must comply with GDPR and CCPA data subject rights: users can request export of their workspace data and request deletion of workspace data. [14][15]
- Creator personal data (email, phone) must be deletable per workspace and globally if a creator exercises a right-to-erasure request.
- All creator data is scraped from public sources. The platform must not access private or authenticated data on any social media platform.
- A Data Processing Agreement (DPA) must be available for enterprise customers (planned for S3).
- All legal artifacts (Terms of Service, Privacy Policy, DPA) require qualified legal counsel review before publication. Pakistan counsel is required for entity and PECA matters; EU/US counsel for GDPR/CCPA.

6.4 Data Retention

- Raw scraped payloads are retained for a defined window (specific duration to be confirmed by legal — see Appendix C, TBD-001).
- Workspace data is retained 30 days after account cancellation, then deleted. Users can export during this window.
- Financial ledger records are retained for the applicable financial retention period (7 years in most jurisdictions) even after workspace deletion.

The following terms are used throughout this document and across all MUSHIN engineering documents:

Creator: A person who creates content on social media platforms (Instagram, TikTok, YouTube, etc.), also known as an influencer. In MUSHIN, a Creator is a canonical data record that can have multiple linked social profiles.

Profile: A single social media account belonging to a Creator. One Creator can have profiles on multiple platforms (e.g., an Instagram profile and a TikTok profile).

Workspace: The container for a team's data in MUSHIN. A workspace has its own members, lists, campaigns, and billing. Data from one workspace is never visible to another.

List: A named, shareable collection of Creators within a workspace. Lists are used to organize creators being considered for a campaign.

Campaign: A project container in MUSHIN for managing a creator marketing initiative from brief to results. Contains a pipeline, budget, tasks, outreach threads, and performance data.

Credit: The internal unit of usage in MUSHIN. Credits are spent on actions that have real API costs: enriching a creator profile, revealing contact info, running a live search, generating AI summaries. Credits are purchased via subscription or top-up packs.

Entitlement: The set of features and limits a workspace is allowed to use, based on their current subscription. Entitlements are always derived from Paddle's subscription data.

Authenticity Score: An AI-generated score that estimates how genuine a creator's followers and engagement are. Always shown with the evidence that produced it — never as a bare number.

Enrichment: The process of fetching or refreshing data for a creator profile from external sources (scraping public social media data). Enrichment costs credits.

Two Brains: The nickname for MUSHIN's two-part data retrieval system. Brain 1 searches the internal database instantly (fast, free). Brain 2 discovers new creators live by scraping public social media data (slower, costs credits).

Live Discovery: A background job (Brain 2) that finds and scrapes new creators matching a search query. Results appear progressively. The user always confirms before a live discovery job starts.

Outbox: A database table that stores events to be published to the message queue. API handlers write to the outbox (a regular database write), and a separate process reads from it and publishes to the queue. This ensures no events are lost even if the queue is temporarily unavailable.

Adapter: A software component that wraps an external service (Apify, Serper, OpenAI, Paddle, Gmail, etc.). All external calls go through adapters. Business logic never calls external APIs directly.

GCP (Global Creator Plane): The database schema holding all creator and profile data. Shared across all workspaces. Contains zero workspace-specific data.

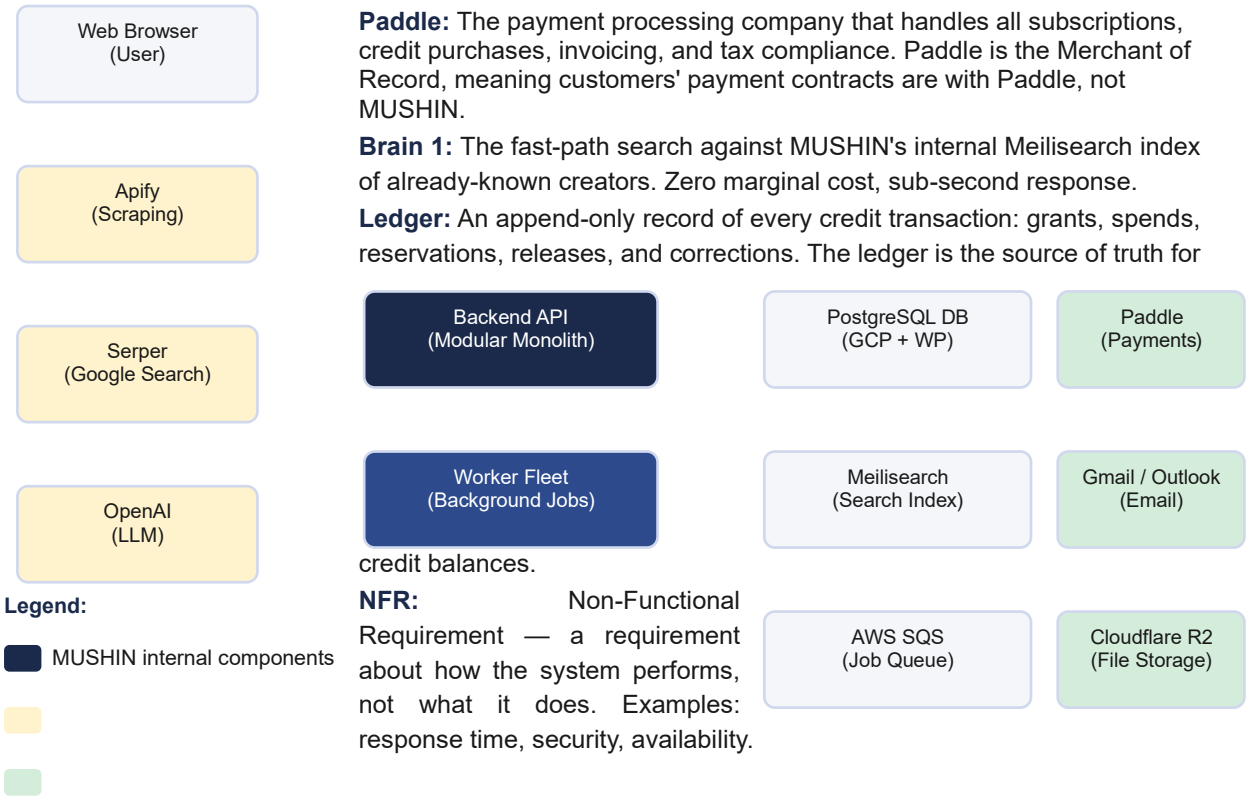
WP (Workspace Plane): The database schema holding all workspace-specific data (lists, campaigns, notes, ledger, consent). Every table has a workspace_id and is protected by Row Level Security.

RLS (Row Level Security): A PostgreSQL feature that enforces data isolation at the database engine level. Even if application code has a bug and forgets to filter by workspace, the database will refuse to return another workspace's data.

MoSCoW: A prioritization framework: Must Have (required for launch), Should Have (important but not blocking), Could Have (nice-to-have), Won't Have now (explicitly deferred).

S1 / S2 / S3: The three launch stages: S1 = MVP launch (design partners), S2 = public launch, S3 = enterprise launch.

ADR: Architecture Decision Record — a document recording a significant technical or product decision, why it was made, and what alternatives were rejected.



Paddle: The payment processing company that handles all subscriptions, credit purchases, invoicing, and tax compliance. Paddle is the Merchant of Record, meaning customers' payment contracts are with Paddle, not MUSHIN.

Brain 1: The fast-path search against MUSHIN's internal Meilisearch index of already-known creators. Zero marginal cost, sub-second response.

Ledger: An append-only record of every credit transaction: grants, spends, reservations, releases, and corrections. The ledger is the source of truth for credit balances.

NFR: Non-Functional Requirement — a requirement about how the system performs, not what it does. Examples: response time, security, availability.

B1. System Context Overview

The diagram below shows MUSHIN's major internal modules and how they connect to external services:

B2. User Journey — Campaign Lifecycle (Primary Flow)

The following is the primary user journey through MUSHIN:

Step	Activity — Old Way !' MUSHIN Way
Step 1: Brief	User creates a campaign and fills in the brief (goals, audience, budget). Old way: spreadsheet template. MUSHIN way: structured form, machine-readable.
Step 2: Discovery	User searches for creators using filters or natural language. Old way: database filters + manual Instagram browsing. MUSHIN way: intelligent ranked search.
Step 3: Vetting	User checks creator authenticity. Old way: separate audit tool + copy-paste. MUSHIN way: in-profile explainable authenticity score.
Step 4: Shortlist	User saves selected creators to a list, shares with client for approval. Old way: export to Sheets + email chaos. MUSHIN way: shared Lists with comments and permissions.
Step 5: Outreach	User contacts creators. Old way: Gmail + a separate templates tool. MUSHIN way: integrated outreach with tracked sequences.
Step 6: Execution	User tracks deliverables and tasks. Old way: Asana + Docs + status emails. MUSHIN way: campaign pipeline with tasks and milestones.
Step 7: Reporting	User produces results report for client. Old way: manual screenshot reports. MUSHIN way: campaign analytics and exportable reports.
Step 8: Memory	Past collaboration data is preserved for the future. Old way: knowledge disappears. MUSHIN way: rate history, notes, and interaction log remain on the creator profile.

The following items are referenced in this SRS but not yet fully decided. They must be resolved before or during the implementation phase they affect:

ID	Description
TBD-001	Exact data retention period for ra yloads (requires legal co nsel sign-off for GDPR compliance). Affects: Section 6 Data Retention
TBD-002	Final credit price per action for each action type (enri henticity score, contact reveal, AI summary). Affects: Sections 4.1–4.3, Billing desig Blocked on Apify/Serper/LLM vendor cost data.
TBD-003	Paddle entity onboarding for Pakistan-registered compan Existential dependency: if Paddle cannot onboard the PK entity, the entire billing model and must be revised.
TBD-004	Pipeline validation results for Apify actor quality on Pakistani/South Asian creator dat This must be validated with a test panel before S1 launch. If the pipeline cannot find a Pakistani creators, the Pakistan-first launch strategy is at risk.
TBD-005	Legal posture memo on public-data scraping from a P isdictio Elevated to high priority following the Two Brains architecture decision. Affects produc claimers.

TBD-006	Exact WCAG 2.1 AA audit scope and delivery milestone. Required for Stage 2 governs detailed accessibility requirements.
TBD-007	Final LLM provider selectio vs. alternatives) and routing policy for the confidence-based escalation ladder. Affects: Section 4.3
TBD-008	Decision on whether credit top-up packs expire and if so, after how many months (12 months is the working assumption). Requires legal counsel review for enforceability in Pakistan and EU jurisdictions.
TBD-009	Default pipeline stages (e.g., "Identified !’ Contacted !’ !’ Confirmed !’ Live !’ Completed"). To be finalized with design partner input. Affects: Section 4.5
TBD-010	Identity resolution rules for matchi creator across Instagram, TikTok, and YouTube accounts (transliteration-aware). Assigned t (Domain Model). This is technically hard and must be resolved before the creator merging feature

This document was prepared from the approved MUSHIN project requirements and specifications. Template structure